

ShuffleNetV2 Bayesian Hyperparameter Optimization

Technical Report

Multi-Objective MOTPE & Hypervolume Maximization
CPU-Only Training & INT8 Post-Training Quantization
MedMNIST PathMNIST (9-class Histopathology)

1. Executive Summary

This project implements an advanced hyperparameter optimization pipeline for ShuffleNetV2 on the MedMNIST PathMNIST dataset (9-class histopathology classification). The pipeline maps the Accuracy vs. Latency vs. Energy Pareto front using two Bayesian Optimization (BO) strategies:

- Multi-Objective Tree-structured Parzen Estimator (MOTPE) -- produces a Pareto-optimal set of configurations across three objectives.
- Hypervolume Maximization -- collapses all three objectives into a single scalar reward (the Hypervolume Indicator) and optimizes it with standard single-objective BO.

All training and inference profiling is CPU-only, and the best configuration is compressed via INT8 Post-Training Quantization (PTQ) for deployment.

2. Problem Statement

Modern edge and embedded deployments require neural networks that balance:

- Accuracy -- classification performance on the target task
- Latency -- per-batch inference time on the target CPU hardware
- Energy Consumption -- total CPU + RAM energy during inference

The challenge is to find hyperparameter configurations that achieve the best trade-off among these three conflicting objectives. A brute-force grid search over the full space is infeasible (34,560+ discrete combinations), so Bayesian Optimization is used to intelligently explore the space in only 100-150 trials.

3. Architecture: ShuffleNetV2

3.1 Why ShuffleNetV2?

ShuffleNetV2 is a lightweight convolutional neural network designed for efficient inference on resource-constrained devices. Its key innovations:

- Channel Split: Divides input channels into two halves -- one passes through unchanged (identity shortcut), the other is transformed.
- Channel Shuffle: Enables cross-group information flow after group convolutions, eliminating the need for costly 1x1 convolutions.
- Depthwise Separable Convolutions: Reduces FLOPs by factoring spatial and channel-wise convolutions.

3.2 Model Implementation

The model is implemented in two classes:

ShuffleNetV2 (models/shufflenet.py):

- 4 stages: stem conv (stride=1 for small 28x28 inputs) -> 3 stacked

ShuffleV2Block stages

- Width multiplier controls channel counts: 0.5x (48->192), 1.0x (116->464), 1.5x (176->704), 2.0x (244->976)
- Global average pooling -> Dropout -> Linear classifier (9 classes)
- Kaiming initialization for all conv layers

QuantizableShuffleNetV2 (subclass):

- Adds QuantStub / DeQuantStub for static quantization
- fuse_model() method fuses Conv2d + BatchNorm2d + ReLU for each stage

ShuffleV2Block (models/blocks.py):

- Stride=1: Channel split -> right branch (1x1 -> DW 3x3 -> 1x1) -> concatenate -> channel shuffle
- Stride=2: Both branches downsample -> concatenate -> channel shuffle

3.3 Parameter Counts by Width Multiplier

Width	Stage Channels	Total Params (approx.)
0.5x	24->48->96->192->1024	~350K
1.0x	24->116->232->464->1024	~1.3M
1.5x	24->176->352->704->1024	~2.5M
2.0x	24->244->488->976->2048	~5.4M

4. Dataset: MedMNIST PathMNIST

- Task: 9-class colorectal cancer histopathology classification
- Images: 28x28 RGB patches from colorectal cancer tissue
- Split: 89,996 train / 10,004 validation / 7,180 test
- Preprocessing: Resize to input_size x input_size -> ToTensor -> ImageNet normalization (mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
- Data loading: num_workers=0, pin_memory=False (conservative, avoids thread contention during parallel BO trials)

5. Hyperparameter Search Space

The search space was expanded from the original 72-configuration grid search to a 34,560+ discrete combination space with 10 hyperparameters.

5.1 Discrete (Categorical) Hyperparameters

Parameter	Choices	Count	Rationale
`width_multiplier`	[0.5, 1.0, 1.5, 2.0]	4	Primary capacity axis -- controls model s
`input_resolution`	[20, 24, 28, 32]	4	Quadratic FLOPs scaling; smaller = faste
`batch_size`	[4, 8, 16, 32, 64, 128]	6	Cache locality, memory pressure, gradie
`intra_op_threads`	[1, 2, 4, 8]	4	CPU thread pool size; affects latency an
`dropout`	[0.0, 0.1, 0.2, 0.3, 0.5]	5	Regularization before classifier head
`optimizer_name`	[adam, sgd, rmsprop]	3	Optimization trajectory and convergence
`scheduler_name`	[cosine, step, onecycle]	3	Learning rate annealing strategy

5.2 Continuous Hyperparameters

Parameter	Range	Distribution	Rationale
`learning_rate`	[1e-5, 3e-1]	Log-uniform	Spans 4 orders of magnitude
`weight_decay`	[1e-6, 1e-2]	Log-uniform	L2 regularization strength
`label_smoothing`	[0.0, 0.20]	Uniform	Soft-target regularization
`momentum`	[0.80, 0.99]	Uniform	Only active for SGD/RMSprop

5.3 Total Search Space

Discrete combos: 4 x 4 x 6 x 4 x 5 x 3 x 3 = 34,560

Plus 4 continuous dimensions -> effectively infinite.

BO explores this with only 100-150 trials.

6. Training Engine

6.1 CPU-Only Training (`engine/trainer.py`)

All training runs on CPU with torch.set_num_threads() controlling parallelism.

GPU/CUDA code paths have been completely removed for consistent, reproducible

CPU energy and latency measurements.

6.2 Optimizer Selection

The build_optimizer() function supports three optimizers:

Optimizer	Characteristics	When Useful
Adam	Adaptive LR per-parameter, momentum-free	Default; good for most tasks
SGD + Nesterov	Classical momentum-based; uses `cfg.mom	Better generalization with tuned LR
RMSprop	Adaptive with momentum; uses `cfg.moment	Good for noisy gradients

6.3 Learning Rate Schedulers

The build_scheduler() function supports three strategies:

Scheduler	Behavior	Best For
CosineAnnealingLR	Smooth cosine decay over `num_epochs`	Standard training
StepLR	Multiplicative decay (x0.5) every `num_epoc	Longer training runs
OneCycleLR	Warm-up -> peak -> cooldown per step	Fast convergence, short runs

6.4 Label Smoothing

CrossEntropyLoss with `label_smoothing=e` replaces hard one-hot targets with soft targets: $(1-e) \cdot \text{one_hot} + e/K$, reducing overconfidence and improving generalization.

7. Optimization Strategies

7.1 Mode 1: Multi-Objective MOTPE (`optuna\_optimize.py`)

Algorithm: Optuna's Multi-Objective Tree-structured Parzen Estimator (MOTPE)

Objectives (optimized simultaneously):

- Maximize Validation Accuracy (%)
- Minimize Inference Latency (seconds per batch)
- Minimize Energy Consumption (kWh, CPU + RAM)

Workflow per trial:

- Optuna's TPESampler suggests a hyperparameter configuration
- Model is trained for `search_epochs` (default: 2) on CPU
- Model is moved to CPU; thread count is set to the trial's `intra_op_threads`
- 3 warm-up forward passes are run (discard thread init spikes)
- Full test set evaluation under CodeCarbon energy tracking
- (accuracy, latency, energy) triplet is returned to Optuna
- Trial checkpoint is saved to `checkpoints/trial_N.pth`

Post-Search:

- Optuna identifies the Pareto-optimal set (non-dominated configurations)
- The trial with the highest accuracy on the Pareto front is selected
- The best model is saved and passed through INT8 PTQ

Command:

```
python3 main.py --trials 150 --optuna-epochs 2
```

7.2 Mode 2: Hypervolume Maximization (`hypervolume\_optimize.py`)

Concept: Instead of multi-objective optimization, frame it as a single-objective problem by maximizing the Hypervolume Indicator (HV).

What is Hypervolume?

$HV(S, r)$ = the volume of objective space dominated by the Pareto front of

solution set S, bounded by reference point r. A higher HV means the Pareto front is both closer to the ideal point AND more spread out.

Reference Point Strategy:

Dynamic nadir point computed from the worst observed values across all completed trials, with a 10% slack buffer to ensure dominated coverage.

HV Computation:

- Primary: Optuna's built-in WFG (Walking Fish Group) algorithm
- Fallback: Monte Carlo approximation (50,000 samples) for older Optuna versions

Why HV Maximization?

Maximizing HV simultaneously encourages:

- High accuracy (Pareto spread in accuracy axis)
- Low latency (Pareto spread in latency axis)
- Low energy (Pareto spread in energy axis)

...all through a single unified TPE surrogate model. This is conceptually simpler than multi-objective optimization and can be more sample-efficient.

Command:

```
python3 main.py --hypervolume --trials 150 --optuna-epochs 2
```

7.3 Comparison of Modes

Aspect	MOTPE	HV Maximization
Objectives	3 separate	1 scalar (HV)
Output	Pareto front (many configs)	Single best config
BO Solver	TPESampler (multi-obj)	TPESampler (single-obj)
Interpretability	Trade-off curves	Convergence plot
When to use	Exploring trade-offs	Deploying one model

8. Energy & Latency Profiling

8.1 CodeCarbon Integration

Energy measurement uses the CodeCarbon library's OfflineEmissionsTracker:

- Tracks CPU energy (via Intel RAPL or PowerCap) and RAM energy
- Reports total energy in kWh
- Country ISO code set to "USA" for emissions factor

8.2 Profiler Module (`engine/profiler.py`)

Isolated benchmarking (used post-search for final evaluation):

- Disable gradients, enable MKL-DNN
- Build dummy input matching inference shape

- Run warm-up passes (discard cold-cache overhead)
- Time and energy-track N forward passes under CodeCarbon
- Report: energy (kWh), average latency (ms), throughput (samples/s)

8.3 In-Trial Profiling

During BO trials, latency and energy are measured over the full test set:

- More realistic than dummy-input profiling
- Includes variable batch composition
- Latency = total test time / number of batches

9. Post-Training Quantization (PTQ)

9.1 Workflow

- Module Fusion: Conv2d + BatchNorm2d + ReLU fused into single ops
(eliminates BN overhead, reduces memory traffic)
- Observer Insertion: Quantization observers track activation ranges
- Calibration: Run 2 test batches through the prepared model
- Conversion: Convert to INT8 using fbgemm (x86 CPU) backend
- Footprint Comparison: Print float vs. INT8 model size in MB

9.2 Quantization Configuration

- Backend: fbgemm (optimized for x86 CPUs)
- Observer: Histogram-based (default for fbgemm)
- Calibration: 2 batches from test set
- Fusion targets: All Conv+BN+ReLU in stage1, stages 2-4 blocks, and conv5

9.3 Expected Results

- Model size reduction: Typically 3-4x compression (FP32 -> INT8)
- Latency improvement: 1.5-2x speedup on x86 CPUs with VNNI/AVX-512
- Accuracy: Minimal degradation (<1%) with proper calibration

10. Project Structure

```
shufflenet_tuning/
??? configs/
?   ??? experiment_config.py      # ExperimentConfig dataclass (11 hyperparameters)
?   ??? base_config.py           # 34,560-combo grid + helper functions
?
??? models/
?   ??? blocks.py                # ShuffleV2Block (channel shuffle + split)
?   ??? shufflenet.py            # ShuffleNetV2 + QuantizableShuffleNetV2
```

```

?
??? engine/
?   ??? trainer.py           # CPU-only training (3 optimizers, 3 schedulers)
?   ??? evaluator.py         # Top-1 accuracy evaluation
?   ??? profiler.py          # Isolated CPU energy + latency benchmarking
?
??? experiments/
?   ??? optuna_optimize.py    # Multi-Objective MOTPE BO + INT8 PTQ
?   ??? hypervolume_optimize.py # Single-Objective HV Maximization BO + PTQ
?   ??? generate_report.py    # CSV, plots, and Markdown report generation
?   ??? train_phase1.py       # Legacy grid-search training
?   ??? profile_phase2.py     # Legacy CPU profiling
?
??? main.py                  # Unified CLI entrypoint
??? checkpoints/             # Saved model weights (float + quantized)
??? results/                 # Generated CSV, plots, and reports

```

11. CLI Reference

Multi-Objective MOTPE (default)

```
python3 main.py --trials 150 --optuna-epochs 2 --final-epochs 8
```

Hypervolume Maximization

```
python3 main.py --hypervolume --trials 150 --optuna-epochs 2
```

Generate Report & Plots

```
python3 experiments/generate_report.py
```

Key Flags

Flag	Default	Description
`--trials`	150	Number of BO trials
`--optuna-epochs`	2	Training epochs per trial
`--final-epochs`	8	Retraining epochs for best config
`--hypervolume`	false	Use HV Maximization mode
`--workers`	1	Parallel Optuna workers
`--log-file`	run.log	Log file path

12. Generated Outputs

File	Description
`results/final_study_results.csv`	All trials with 11 hyperparameters + 3 objectives
`results/accuracy_vs_latency.png`	2D Pareto slice (width-colored)

`results/accuracy_vs_energy.png`	2D Pareto slice (width-colored)
`results/pareto_3d.png`	Full 3D Pareto front scatter
`results/param_importance.png`	FAnova hyperparameter importance
`results/hv_convergence.png`	HV indicator vs. trial number
`results/ShuffleNetV2_BO_Report.md`	Text summary of results
`checkpoints/best_model_float.pth`	Best FP32 model weights
`checkpoints/best_model_quantized.pth`	Best INT8 quantized weights

13. Dependencies

Package	Version	Purpose
`torch`	>=1.13.0	Deep learning framework, quantization APIs
`torchvision`	>=0.14.0	Image transforms (Resize, Normalize)
`medmnist`	>=2.1.0	PathMNIST dataset
`optuna`	>=3.0.0	Bayesian optimization (TPESampler, MOTPE)
`codecarbon`	>=2.1.0	CPU energy tracking (RAPL/PowerCap)
`matplotlib`	>=3.5.0	Pareto plots, convergence curves
`numpy`	>=1.21.0	Numerical operations
`pandas`	>=1.3.0	Data analysis
`pysqlite3`	>=0.5.0	SQLite backend for Optuna study storage

14. Key Design Decisions

- CPU-Only Training: All GPU paths removed to ensure latency/energy measurements are consistent and reproducible on the target deployment hardware.
- Expanded Search Space: From 72 grid combos to 34,560+ discrete combos with 4 additional hyperparameters (optimizer, scheduler, label smoothing, momentum) plus widened ranges.
- BO over Grid Search: With 34,560+ combos, grid search is infeasible. TPE-based BO evaluates only 100-150 trials by building a probabilistic surrogate of the objective landscape.
- Dual BO Modes: MOTPE for full Pareto exploration; HV Maximization for when a single best configuration is needed.
- Conservative Data Loading: num_workers=0 and pin_memory=False to avoid thread contention when Optuna runs parallel trials.
- Dynamic Reference Point for HV: Computed from worst observed values with 10% slack, ensuring the reference point is always dominated.

All code is CPU-oriented and designed for deployment on resource-constrained hardware.